Team - NEXUS

BIZPILOT

B2B AI Business Assistant with WhatsApp Customer Bots

SOFTWARE DESIGN DOCUMENT

Lead By: Anwesha Arya
Prepared By: Team

Contents

1. Overview
2. Scope and Core Requirements
3. System Architecture
4. Detailed Workflows
5. Data Design
6. Interfaces
7. Security, Reliability and AI Safeguards
8. Testing and Rollout
9. Risks and Open Decisions

1. Overview

BizPilot is a B2B AI business assistant for shop owners, retailers, and MSMEs. Business owners use a web dashboard to manage products, inventory, orders, billing, approvals, and insights. Their customers interact only through the business's WhatsApp bot and do not create BizPilot accounts.

Design goals

- Use verified business data for customer enquiries and business actions.
- Keep customer names, phone numbers, addresses, and identifiable message content **out of** BizPilot persistent storage and logs.
- Use backend rules for calculations, permissions, inventory updates, and approvals.

2. Scope and Core Requirements

- Owner authentication, business profile, products, selling prices, stock, and reorder thresholds.
- WhatsApp customer enquiries in supported Hindi, English, and Hinglish; configured FAQs and order requests.
- Inventory alerts, reorder suggestions, slow-moving stock insights, invoices, daily/weekly sales summaries, and owner approvals.
- Customer interaction is WhatsApp-only. No customer-facing BizPilot dashboard, login.

3. System Architecture

Use a modular backend for the initial release. Keep owner operations and the WhatsApp customer channel as separate interfaces, with business data isolated by tenant.

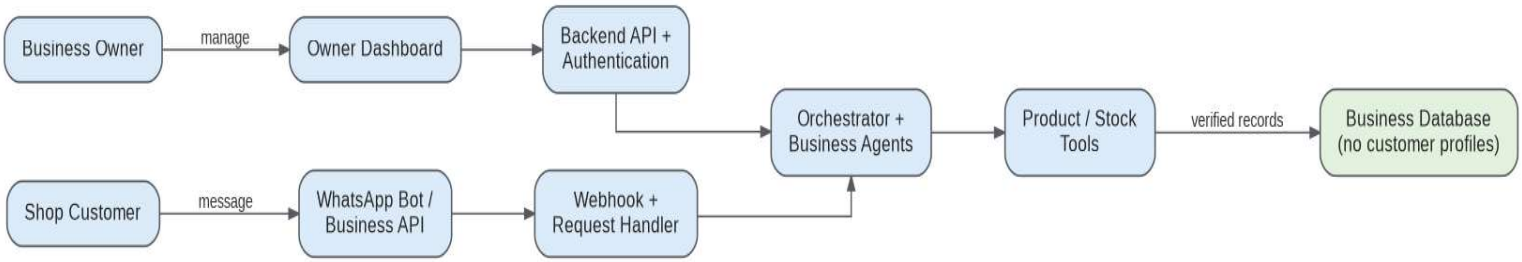


Figure 1. Owner operations and WhatsApp customer flow.

Main components

- Owner dashboard and backend API/authentication.
- WhatsApp Business adapter and webhook validation.
- Manager/Orchestrator Agent and specialized Customer, Inventory, Billing, and Insights Agents.
- Authorized business tools, validation/approval gate, MongoDB business records, and redacted monitoring.

Technology choices

Area	Technology / role
Owner frontend	React dashboard
Backend	Node.js + Express
Database	PostgreSQL
AI orchestration	Python + FastAPI + LangGraph, Gemini
Messaging	WhatsApp Business Platform / approved provider
Invoice output	Backend PDF generation

4. Detailed Workflows

4.1 WhatsApp product enquiry

- Customer sends a WhatsApp message to the business bot.
- Webhook validates the provider event and resolves the business from the receiving business number.
- Orchestrator routes the request to the Customer Agent.
- Agent retrieves recorded product, price, and stock data through authorized tools.
- Backend validates the result and sends a supported-language reply.
- Discard transient message/context after processing; do not retain customer identity or message content.

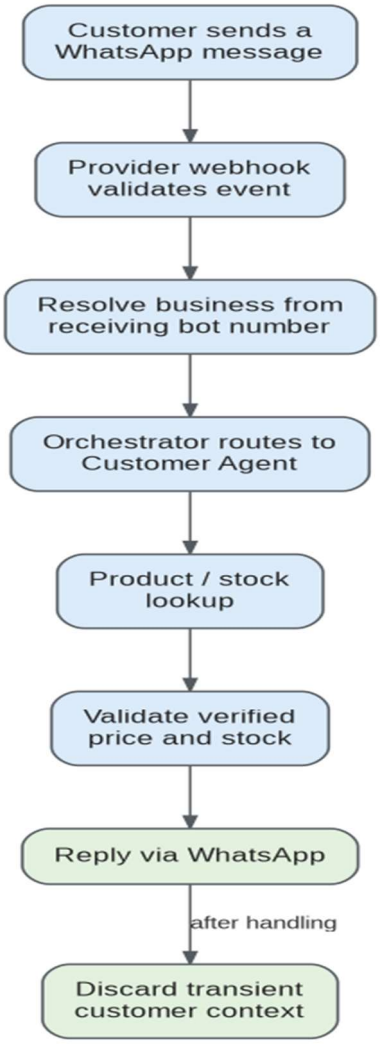


Figure 2. Customer enquiry workflow.

4.2 Order request and approval

- Receive requested items and quantities through WhatsApp.
- Validate products, quantities, and stock; reject or clarify if stock is insufficient.
- Calculate totals with backend rules and create a business order reference without customer identity.
- If required, hold the action for owner approval.
- On approval, recheck stock and commit the order/inventory update safely; otherwise do not execute.
- Record operational outcome without customer-identifying details.

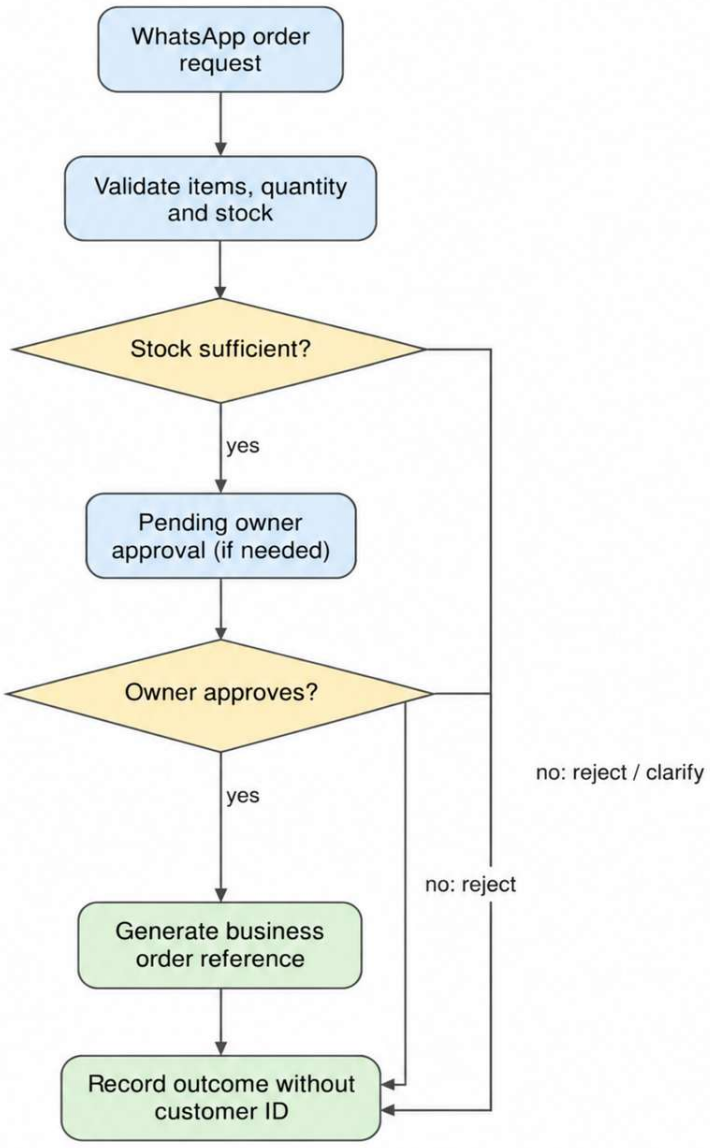


Figure 3. Order workflow with approval and privacy boundary.

4.3 Invoice generation

- Load validated order and business details.
- Check required invoice fields and verified tax inputs.
- Calculate line totals, discounts, taxes, and total using deterministic backend code.
- Hold for review if tax treatment or required information is unclear; apply configured approval rules.
- Store invoice number, order reference, totals, status, and timestamp; generate readable output/PDF.

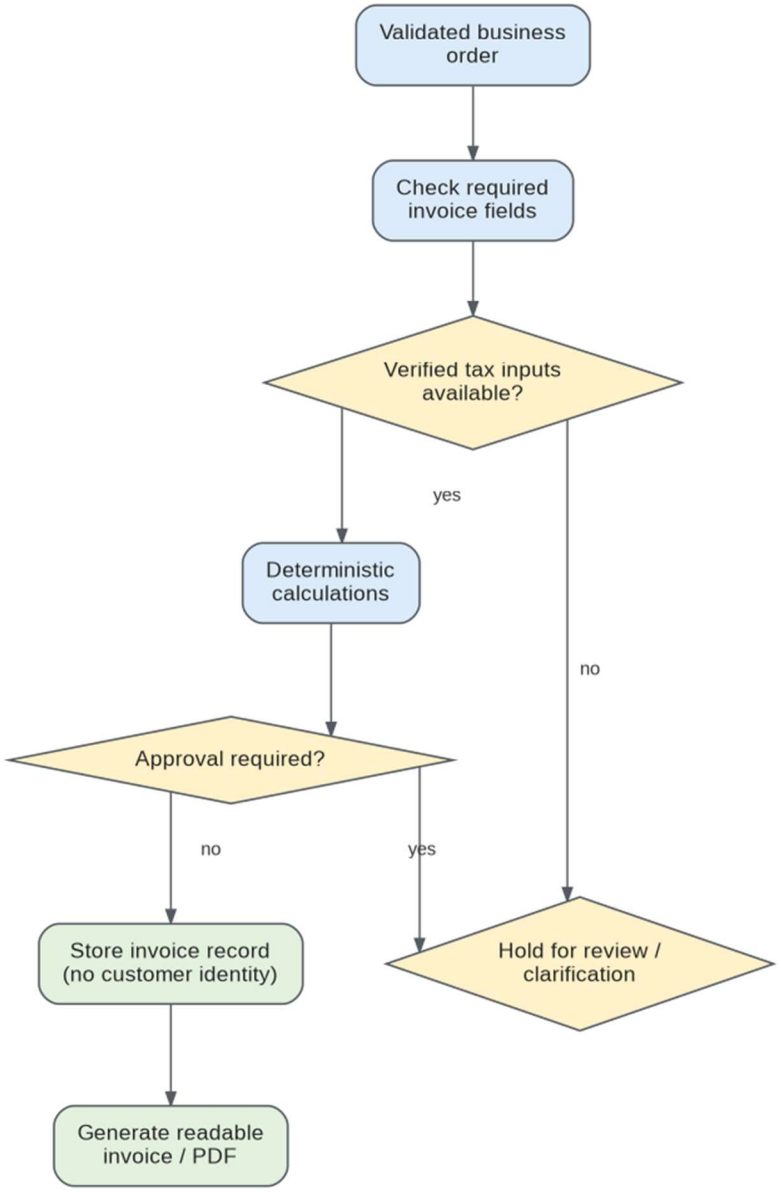


Figure 4. Invoice flow.

Business rules

- Do not oversell unless a separately authorized backorder flow is defined.
- Low-stock alert when quantity is at or below configured threshold.
- Line total = quantity × recorded unit selling price, subject to verified discounts and tax rules.
- Never guess tax rates or claim success after a failed operation.
- Use idempotency and safe retries to prevent duplicate orders/invoices.

5. Data Design

BizPilot stores business and operational records only. Do not create a CUSTOMER collection or persist customer identity/message content in orders, invoices, audit logs, analytics, or agent traces.

Record	Stored data (customer identity excluded)
BUSINESS	business_id, name, owner_user_id, settings
PRODUCT / INVENTORY	product_id, business_id, name/SKU, selling price, quantity, reorder threshold
ORDER	order_id, business_id, items, quantities, transaction-time prices, total, status, timestamp
INVOICE	invoice_id, business_id, order reference, invoice number, totals, tax breakdown, status, timestamp
APPROVAL	approval_id, business_id, action reference, decision, approver, timestamp
AUDIT / AGENT EXECUTION	business/request reference, action/tool outcome, latency, error code; no raw messages or customer identifiers

Privacy and retention

- Process sender identifiers and message text only as transient input needed to handle the active request.
- Do not persist customer names, phone numbers, addresses, sender IDs, or raw message content in BizPilot storage or logs.
- Minimize/redact inputs sent to AI services; configure webhook payloads and traces to avoid unnecessary retention.
- Invoice/legal requirements involving customer details must be confirmed before such workflows are enabled; this design excludes those fields by default.

6. Interfaces

- Owner dashboard ↔ backend API: business settings, products, inventory, orders, invoices, approvals, reports.
- WhatsApp provider → webhook: customer messages and delivery callbacks.
- Backend ↔ WhatsApp provider: bot replies.
- Orchestrator ↔ agents/tools: structured requests, verified lookups, deterministic calculations, and authorized actions.
- Backend ↔ database: business and transaction records, no customer profile store.

7. Security, Reliability, and AI Safeguards

- Enforce owner authentication, role permissions, and tenant isolation in the backend.
- Treat customer messages as untrusted, prompts cannot override permissions or business rules.
- Limit each agent to its purpose, tools, and required context; validate structured outputs.
- Use protected secrets, webhook validation, rate limits, bounded retries, event deduplication, and idempotent operations.
- Logs and monitoring capture routing, outcomes, latency, and errors while excluding customer identity and raw messages.

8. Testing and Rollout

- Test product answers against verified stock and price records.
- Test webhook authenticity, duplicate events, provider failures, and WhatsApp reply handling.
- Test inventory thresholds, stock rejection, approvals, invoice arithmetic, and missing tax inputs.
- Verify no customer identity/message content is present in database, logs, analytics, or agent traces.
- Test access isolation, prompt injection, supported-language requests, and retry safety.
- Prototype owner dashboard and business records with test data.
- Implement WhatsApp webhook and product enquiry path.
- Add order handling, inventory updates, approval, and billing.
- Integrate provider sandbox and complete privacy/security tests before pilot.

9. Risks and Open Decisions

- Customer data may leak through provider payloads, logs, or AI traces: audit every path and add automated privacy tests.
- WhatsApp access/policies and outages: validate provider requirements and provide bounded retries/fallback.
- Tax or invoice requirements may conflict with customer-data minimization: confirm requirements before production billing.
- Confirm approval thresholds, retention/backups, and operational targets with the project team.